Audit:

List the policies in use for each namespace in the cluster, ensure that each policy disallows the admission of privileged containers.
Since manually searching through each pod's configuration might be tedious, especially in environments with many pods, you can use a more automated approach with grep or other command-line tools.
Here's an example of how you might approach this with a combination of kubectl, grep, and shell scripting for a more automated solution:

```
kubectl get pods --all-namespaces -o json | jq -r '.items[] |
select(.spec.containers[].securityContext.privileged == true) |
.metadata.name'
```

OR

```
kubectl get pods --all-namespaces -o json | jq '.items[] |
select(.metadata.namespace != "kube-system" and
.spec.containers[]?.securityContext?.privileged == true) | {pod:
.metadata.name, namespace: .metadata.namespace, container:
.spec.containers[].name}'
```

When creating a Pod Security Policy, ["kube-system"] namespaces are excluded by default.
This command uses jq, a command-line JSON processor, to parse the JSON output from kubectl get pods and filter out pods where any container has the securityContext.privileged flag set to true. Please note that you might need to adjust the command depending on your specific requirements and the structure of your pod specifications.

Remediation:

Add policies to each namespace in the cluster which has user workloads to restrict the admission of privileged containers.
To enable PSA for a namespace in your cluster, set the pod-security.kubernetes.io/enforce label with the policy value you want to enforce.

```
kubectl label --overwrite ns NAMESPACE pod-
security.kubernetes.io/enforce=restricted
```

The above command enforces the restricted policy for the NAMESPACE namespace.
You can also enable Pod Security Admission for all your namespaces. For example:

```
kubectl label --overwrite ns --all pod-
security.kubernetes.io/warn=baseline
```

Default Value:

By default, there are no restrictions on the creation of privileged containers.

References:

1. https://kubernetes.io/docs/concepts/security/pod-security-admission/
2. https://aws.github.io/aws-eks-best-practices/pods/#restrict-the-containers-that-can-run-as-privileged

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	5.4 Restrict Administrator Privileges to Dedicated Administrator Accounts Restrict administrator privileges to dedicated administrator accounts on enterprise assets. Conduct general computing activities, such as internet browsing, email, and productivity suite use, from the user's primary, non-privileged account.	●	●	●
v7	4.3 Ensure the Use of Dedicated Administrative Accounts Ensure that all users with administrative account access use a dedicated or secondary account for elevated activities. This account should only be used for administrative activities and not internet browsing, email, or similar activities.	●	●	●

4.2.2 Minimize the admission of containers wishing to share the host process ID namespace (Automated)

Profile Applicability:

- Level 1

Description:

Do not generally permit containers to be run with the `hostPID` flag set to true.

Rationale:

A container running in the host's PID namespace can inspect processes running outside the container. If the container also has access to ptrace capabilities this can be used to escalate privileges outside of the container.

There should be at least one admission control policy defined which does not permit containers to share the host PID namespace.

If you need to run containers which require hostPID, this should be defined in a separate policy and you should carefully check to ensure that only limited service accounts and users are given permission to use that policy.

Impact:

Pods defined with `spec.hostPID: true` will not be permitted unless they are run under a specific policy.

Audit:

List the policies in use for each namespace in the cluster, ensure that each policy disallows the admission of `hostPID` containers
Search for the hostPID Flag: In the YAML output, look for the `hostPID` setting under the spec section to check if it is set to `true`.

```
kubectl get pods --all-namespaces -o json | jq -r '.items[] |
select(.spec.hostPID == true) |
"\(.metadata.namespace)/\(.metadata.name)"'
```

OR

```
kubectl get pods --all-namespaces -o json | jq '.items[] |
select(.metadata.namespace != "kube-system" and .spec.hostPID == true)
| {pod: .metadata.name, namespace: .metadata.namespace, container:
.spec.containers[].name}'
```

When creating a Pod Security Policy, ["kube-system"] namespaces are excluded by default.
This command retrieves all pods across all namespaces in JSON format, then uses jq to filter out those with the `hostPID` flag set to `true`, and finally formats the output to show the namespace and name of each matching pod.

Remediation:

Add policies to each namespace in the cluster which has user workloads to restrict the admission of `hostPID` containers.

Default Value:

By default, there are no restrictions on the creation of `hostPID` containers.

References:

1. https://kubernetes.io/docs/concepts/security/pod-security-admission/

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	5.4 Restrict Administrator Privileges to Dedicated Administrator Accounts Restrict administrator privileges to dedicated administrator accounts on enterprise assets. Conduct general computing activities, such as internet browsing, email, and productivity suite use, from the user's primary, non-privileged account.	●	●	●
v7	4.3 Ensure the Use of Dedicated Administrative Accounts Ensure that all users with administrative account access use a dedicated or secondary account for elevated activities. This account should only be used for administrative activities and not internet browsing, email, or similar activities.	●	●	●

4.2.3 Minimize the admission of containers wishing to share the host IPC namespace (Automated)

Profile Applicability:

- Level 1

Description:

Do not generally permit containers to be run with the `hostIPC` flag set to true.

Rationale:

A container running in the host's IPC namespace can use IPC to interact with processes outside the container.

There should be at least one admission control policy defined which does not permit containers to share the host IPC namespace.

If you need to run containers which require hostIPC, this should be defined in a separate policy and you should carefully check to ensure that only limited service accounts and users are given permission to use that policy.

Impact:

Pods defined with `spec.hostIPC: true` will not be permitted unless they are run under a specific policy.

Audit:

List the policies in use for each namespace in the cluster, ensure that each policy disallows the admission of `hostIPC` containers
Search for the hostIPC Flag: In the YAML output, look for the `hostIPC` setting under the spec section to check if it is set to `true`.

```
kubectl get pods --all-namespaces -o json | jq -r '.items[] |
select(.spec.hostIPC == true) |
"\(.metadata.namespace)/\(.metadata.name)"'
```

OR

```
kubectl get pods --all-namespaces -o json | jq '.items[] |
select(.metadata.namespace != "kube-system" and .spec.hostIPC == true)
| {pod: .metadata.name, namespace: .metadata.namespace, container:
.spec.containers[].name}'
```

When creating a Pod Security Policy, ["kube-system"] namespaces are excluded by default.
This command retrieves all pods across all namespaces in JSON format, then uses jq to filter out those with the `hostIPC` flag set to `true`, and finally formats the output to show the namespace and name of each matching pod.

Remediation:

Add policies to each namespace in the cluster which has user workloads to restrict the admission of `hostIPC` containers.

Default Value:

By default, there are no restrictions on the creation of `hostIPC` containers.

References:

1. https://kubernetes.io/docs/concepts/security/pod-security-admission/

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	5.4 Restrict Administrator Privileges to Dedicated Administrator Accounts Restrict administrator privileges to dedicated administrator accounts on enterprise assets. Conduct general computing activities, such as internet browsing, email, and productivity suite use, from the user's primary, non-privileged account.	●	●	●
v7	4.3 Ensure the Use of Dedicated Administrative Accounts Ensure that all users with administrative account access use a dedicated or secondary account for elevated activities. This account should only be used for administrative activities and not internet browsing, email, or similar activities.	●	●	●

4.2.4 Minimize the admission of containers wishing to share the host network namespace (Automated)

Profile Applicability:

- Level 1

Description:

Do not generally permit containers to be run with the `hostNetwork` flag set to true.

Rationale:

A container running in the host's network namespace could access the local loopback device, and could access network traffic to and from other pods.

There should be at least one admission control policy defined which does not permit containers to share the host network namespace.

If you need to run containers which require access to the host's network namespaces, this should be defined in a separate policy and you should carefully check to ensure that only limited service accounts and users are given permission to use that policy.

Impact:

Pods defined with `spec.hostNetwork: true` will not be permitted unless they are run under a specific policy.

Audit:

List the policies in use for each namespace in the cluster, ensure that each policy disallows the admission of `hostNetwork` containers
Given that manually checking each pod can be time-consuming, especially in large environments, you can use a more automated approach to filter out pods where `hostNetwork` is set to `true`. Here's a command using kubectl and jq:

```
kubectl get pods --all-namespaces -o json | jq -r '.items[] |
select(.spec.hostNetwork == true) |
"\(.metadata.namespace)/\(.metadata.name)"'
```

OR

```
kubectl get pods --all-namespaces -o json | jq '.items[] |
select(.metadata.namespace != "kube-system" and .spec.hostNetwork ==
true) | {pod: .metadata.name, namespace: .metadata.namespace,
container: .spec.containers[].name}'
```

When creating a Pod Security Policy, ["kube-system"] namespaces are excluded by default.
This command retrieves all pods across all namespaces in JSON format, then uses jq to filter out those with the `hostNetwork` flag set to `true`, and finally formats the output to show the namespace and name of each matching pod.

Remediation:

Add policies to each namespace in the cluster which has user workloads to restrict the admission of `hostNetwork` containers.

Default Value:

By default, there are no restrictions on the creation of `hostNetwork` containers.

References:

1. https://kubernetes.io/docs/concepts/security/pod-security-admission/

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	5.4 Restrict Administrator Privileges to Dedicated Administrator Accounts Restrict administrator privileges to dedicated administrator accounts on enterprise assets. Conduct general computing activities, such as internet browsing, email, and productivity suite use, from the user's primary, non-privileged account.	●	●	●
v7	4.3 Ensure the Use of Dedicated Administrative Accounts Ensure that all users with administrative account access use a dedicated or secondary account for elevated activities. This account should only be used for administrative activities and not internet browsing, email, or similar activities.	●	●	●

4.2.5 Minimize the admission of containers with allowPrivilegeEscalation (Automated)

Profile Applicability:

- Level 1

Description:

Do not generally permit containers to be run with the `allowPrivilegeEscalation` flag set to `true`. Allowing this right can lead to a process running a container getting more rights than it started with.

It's important to note that these rights are still constrained by the overall container sandbox, and this setting does not relate to the use of privileged containers.

Rationale:

A container running with the `allowPrivilegeEscalation` flag set to `true` may have processes that can gain more privileges than their parent.

There should be at least one admission control policy defined which does not permit containers to allow privilege escalation. The option exists (and is defaulted to true) to permit setuid binaries to run.

If you have need to run containers which use setuid binaries or require privilege escalation, this should be defined in a separate policy and you should carefully check to ensure that only limited service accounts and users are given permission to use that policy.

Impact:

Pods defined with `spec.allowPrivilegeEscalation: true` will not be permitted unless they are run under a specific policy.

Audit:

List the policies in use for each namespace in the cluster, ensure that each policy disallows the admission of containers which allow privilege escalation.
This command gets all pods across all namespaces, outputs their details in JSON format, and uses jq to parse and filter the output for containers with `allowPrivilegeEscalation` set to `true`.

```
kubectl get pods --all-namespaces -o json | jq -r '.items[] |
select(any(.spec.containers[];
.securityContext.allowPrivilegeEscalation == true)) |
"\(.metadata.namespace)/\(.metadata.name)"'
```

OR

```
kubectl get pods --all-namespaces -o json | jq '.items[] |
select(.metadata.namespace != "kube-system" and .spec.containers[];
.securityContext.allowPrivilegeEscalation == true) | {pod:
.metadata.name, namespace: .metadata.namespace, container:
.spec.containers[].name}'
```

When creating a Pod Security Policy, ["kube-system"] namespaces are excluded by default.
This command uses jq, a command-line JSON processor, to parse the JSON output from kubectl get pods and filter out pods where any container has the securityContext.privileged flag set to true. Please note that you might need to adjust the command depending on your specific requirements and the structure of your pod specifications.

Remediation:

Add policies to each namespace in the cluster which has user workloads to restrict the admission of containers with `.spec.allowPrivilegeEscalation` set to `true`.

Default Value:

By default, there are no restrictions on contained process ability to escalate privileges, within the context of the container.

References:

1. https://kubernetes.io/docs/concepts/security/pod-security-admission/

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	5.4 Restrict Administrator Privileges to Dedicated Administrator Accounts Restrict administrator privileges to dedicated administrator accounts on enterprise assets. Conduct general computing activities, such as internet browsing, email, and productivity suite use, from the user's primary, non-privileged account.	●	●	●
v7	4.3 Ensure the Use of Dedicated Administrative Accounts Ensure that all users with administrative account access use a dedicated or secondary account for elevated activities. This account should only be used for administrative activities and not internet browsing, email, or similar activities.	●	●	●

4.3 CNI Plugin

4.3.1 Ensure CNI plugin supports network policies. (Manual)

Profile Applicability:

- Level 1

Description:

There are a variety of CNI plugins available for Kubernetes. If the CNI in use does not support Network Policies it may not be possible to effectively restrict traffic in the cluster.

Rationale:

Kubernetes network policies are enforced by the CNI plugin in use. As such it is important to ensure that the CNI plugin supports both Ingress and Egress network policies.

Impact:

None.

Audit:

Review the documentation of CNI plugin in use by the cluster, and confirm that it supports network policies.

Remediation:

As with RBAC policies, network policies should adhere to the policy of least privileged access. Start by creating a deny all policy that restricts all inbound and outbound traffic from a namespace or create a global policy using Calico.

Default Value:

This will depend on the CNI plugin in use.

References:

1. https://kubernetes.io/docs/concepts/extend-kubernetes/compute-storage-net/network-plugins/
2. https://aws.github.io/aws-eks-best-practices/network/

Additional Information:

One example here is Flannel (https://github.com/coreos/flannel) which does not support Network policy unless Calico is also in use.

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	7.4 Perform Automated Application Patch Management Perform application updates on enterprise assets through automated patch management on a monthly, or more frequent, basis.	●	●	●
v7	18.4 Only Use Up-to-date And Trusted Third-Party Components Only use up-to-date and trusted third-party components for the software developed by the organization.		●	●

4.3.2 Ensure that all Namespaces have Network Policies defined (Automated)

Profile Applicability:

- Level 1

Description:

Use network policies to isolate traffic in your cluster network.

Rationale:

Running different applications on the same Kubernetes cluster creates a risk of one compromised application attacking a neighboring application. Network segmentation is important to ensure that containers can communicate only with those they are supposed to. A network policy is a specification of how selections of pods are allowed to communicate with each other and other network endpoints.

Once there is any Network Policy in a namespace selecting a particular pod, that pod will reject any connections that are not allowed by any Network Policy. Other pods in the namespace that are not selected by any Network Policy will continue to accept all traffic"

Impact:

Once there is any Network Policy in a namespace selecting a particular pod, that pod will reject any connections that are not allowed by any Network Policy. Other pods in the namespace that are not selected by any Network Policy will continue to accept all traffic"

Audit:

Run the below command and review the `NetworkPolicy` objects created in the cluster.

```
kubectl get networkpolicy --all-namespaces
```

Ensure that each namespace defined in the cluster has at least one Network Policy.

Remediation:

Follow the documentation and create `NetworkPolicy` objects as you need them.

Default Value:

By default, network policies are not created.

References:

1. https://kubernetes.io/docs/concepts/services-networking/networkpolicies/
2. https://octetz.com/posts/k8s-network-policy-apis

3. https://kubernetes.io/docs/tasks/configure-pod-container/declare-network-policy/

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	4.4 Implement and Manage a Firewall on Servers Implement and manage a firewall on servers, where supported. Example implementations include a virtual firewall, operating system firewall, or a third-party firewall agent.	●	●	●
v7	14.1 Segment the Network Based on Sensitivity Segment the network based on the label or classification level of the information stored on the servers, locate all sensitive information on separated Virtual Local Area Networks (VLANs).		●	●
v7	14.2 Enable Firewall Filtering Between VLANs Enable firewall filtering between VLANs to ensure that only authorized systems are able to communicate with other systems necessary to fulfill their specific responsibilities.		●	●

4.4 Secrets Management

4.4.1 Prefer using secrets as files over secrets as environment variables (Automated)

Profile Applicability:

- Level 1

Description:

Kubernetes supports mounting secrets as data volumes or as environment variables. Minimize the use of environment variable secrets.

Rationale:

It is reasonably common for application code to log out its environment (particularly in the event of an error). This will include any secret values passed in as environment variables, so secrets can easily be exposed to any user or entity who has access to the logs.

Impact:

Application code which expects to read secrets in the form of environment variables would need modification

Audit:

Run the following command to find references to objects which use environment variables defined from secrets.

```
kubectl get all -o jsonpath='{range .items[?(@..secretKeyRef)]} {.kind}
{.metadata.name} {"\n"}{end}' -A
```

Remediation:

If possible, rewrite application code to read secrets from mounted secret files, rather than from environment variables.

Default Value:

By default, secrets are not defined

References:

1. https://kubernetes.io/docs/concepts/configuration/secret/#using-secrets

Additional Information:

Mounting secrets as volumes has the additional benefit that secret values can be updated without restarting the pod

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	3.10 Encrypt Sensitive Data in Transit Encrypt sensitive data in transit. Example implementations can include: Transport Layer Security (TLS) and Open Secure Shell (OpenSSH).		●	●
v7	14.4 Encrypt All Sensitive Information in Transit Encrypt all sensitive information in transit.		●	●
v7	14.8 Encrypt Sensitive Information at Rest Encrypt all sensitive information at rest using a tool that requires a secondary authentication mechanism not integrated into the operating system, in order to access the information.			●

4.4.2 Consider external secret storage (Manual)

Profile Applicability:

- Level 1

Description:

Consider the use of an external secrets storage and management system, instead of using Kubernetes Secrets directly, if you have more complex secret management needs. Ensure the solution requires authentication to access secrets, has auditing of access to and use of secrets, and encrypts secrets. Some solutions also make it easier to rotate secrets.

Rationale:

Kubernetes supports secrets as first-class objects, but care needs to be taken to ensure that access to secrets is carefully limited. Using an external secrets provider can ease the management of access to secrets, especially where secrests are used across both Kubernetes and non-Kubernetes environments.

Impact:

None

Audit:

Review your secrets management implementation.

Remediation:

Refer to the secrets management options offered by your cloud provider or a third-party secrets management solution.

Default Value:

By default, no external secret management is configured.

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	3.11 Encrypt Sensitive Data at Rest Encrypt sensitive data at rest on servers, applications, and databases containing sensitive data. Storage-layer encryption, also known as server-side encryption, meets the minimum requirement of this Safeguard. Additional encryption methods may include application-layer encryption, also known as client-side encryption, where access to the data storage device(s) does not permit access to the plain-text data.		●	●

Controls Version	Control	IG 1	IG 2	IG 3
v7	14.8 Encrypt Sensitive Information at Rest Encrypt all sensitive information at rest using a tool that requires a secondary authentication mechanism not integrated into the operating system, in order to access the information.			●

4.5 General Policies

These policies relate to general cluster management topics, like namespace best practices and policies applied to pod objects in the cluster.

4.5.1 Create administrative boundaries between resources using namespaces (Manual)

Profile Applicability:

- Level 1

Description:

Use namespaces to isolate your Kubernetes objects.

Rationale:

Limiting the scope of user permissions can reduce the impact of mistakes or malicious activities. A Kubernetes namespace allows you to partition created resources into logically named groups. Resources created in one namespace can be hidden from other namespaces. By default, each resource created by a user in an Amazon EKS cluster runs in a default namespace, called `default`. You can create additional namespaces and attach resources and users to them. You can use Kubernetes Authorization plugins to create policies that segregate access to namespace resources between different users.

Impact:

You need to switch between namespaces for administration.

Audit:

Run the below command and review the namespaces created in the cluster.

```
kubectl get namespaces
```

Ensure that these namespaces are the ones you need and are adequately administered as per your requirements.

Remediation:

Follow the documentation and create namespaces for objects in your deployment as you need them.

Default Value:

By default, Kubernetes starts with four initial namespaces:

1. `default` - The default namespace for objects with no other namespace
2. `kube-system` - The namespace for objects created by the Kubernetes system
3. `kube-public` - The namespace for public-readable ConfigMap
4. `kube-node-lease` - The namespace for associated lease object for each node

References:

1. https://kubernetes.io/docs/concepts/overview/working-with-objects/namespaces/
2. http://blog.kubernetes.io/2016/08/security-best-practices-kubernetes-deployment.html

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	12.8 Establish and Maintain Dedicated Computing Resources for All Administrative Work Establish and maintain dedicated computing resources, either physically or logically separated, for all administrative tasks or tasks requiring administrative access. The computing resources should be segmented from the enterprise's primary network and not be allowed internet access.			●
v7	12.1 Maintain an Inventory of Network Boundaries Maintain an up-to-date inventory of all of the organization's network boundaries.	●	●	●

4.5.2 The default namespace should not be used (Automated)

Profile Applicability:

- Level 1

Description:

Kubernetes provides a default namespace, where objects are placed if no namespace is specified for them. Placing objects in this namespace makes application of RBAC and other controls more difficult.

Rationale:

Resources in a Kubernetes cluster should be segregated by namespace, to allow for security controls to be applied at that level and to make it easier to manage resources.

Impact:

None

Audit:

Run this command to list objects in default namespace

```
kubectl get $(kubectl api-resources --verbs=list --namespaced=true -o name |
paste -sd, -) --ignore-not-found -n default
```

The only entries there should be system managed resources such as the `kubernetes` service
OR

```
kubectl get pods -n default
```

Returning `No resources found in default namespace.`

Remediation:

Ensure that namespaces are created to allow for appropriate segregation of Kubernetes resources and that all new resources are created in a specific namespace.

Default Value:

Unless a namespace is specific on object creation, the `default` namespace will be used

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	16.7 Use Standard Hardening Configuration Templates for Application Infrastructure Use standard, industry-recommended hardening configuration templates for application infrastructure components. This includes underlying servers, databases, and web servers, and applies to cloud containers, Platform as a Service (PaaS) components, and SaaS components. Do not allow in-house developed software to weaken configuration hardening.		●	●
v7	5.1 Establish Secure Configurations Maintain documented, standard security configuration standards for all authorized operating systems and software.	●	●	●

5 Managed services

This section consists of security recommendations for the Amazon EKS. These recommendations are applicable for configurations that Amazon EKS customers own and manage.

5.1 Image Registry and Image Scanning

This section contains recommendations relating to container image registries and securing images in those registries, such as Amazon Elastic Container Registry (ECR).

5.1.1 Ensure Image Vulnerability Scanning using Amazon ECR image scanning or a third party provider (Automated)

Profile Applicability:

- Level 1

Description:

Scan images being deployed to Amazon EKS for vulnerabilities.

Rationale:

Vulnerabilities in software packages can be exploited by hackers or malicious users to obtain unauthorized access to local cloud resources. Amazon ECR and other third party products allow images to be scanned for known vulnerabilities.

Impact:

If you are utilizing AWS ECR

The following are common image scan failures. You can view errors like this in the Amazon ECR console by displaying the image details or through the API or AWS CLI by using the DescribeImageScanFindings API.

UnsupportedImageError You may get an UnsupportedImageError error when attempting to scan an image that was built using an operating system that Amazon ECR doesn't support image scanning for. Amazon ECR supports package vulnerability scanning for major versions of Amazon Linux, Amazon Linux 2, Debian, Ubuntu, CentOS, Oracle Linux, Alpine, and RHEL Linux distributions. Amazon ECR does not support scanning images built from the Docker scratch image.

An UNDEFINED severity level is returned You may receive a scan finding that has a severity level of UNDEFINED. The following are the common causes for this:

The vulnerability was not assigned a priority by the CVE source.

The vulnerability was assigned a priority that Amazon ECR did not recognize.

To determine the severity and description of a vulnerability, you can view the CVE directly from the source.

Audit:

Please follow AWS ECS or your 3rd party image scanning provider's guidelines for enabling Image Scanning.

```
aws ecr describe-repositories --repository-names $REPO_NAME --region
$REGION_CODE
```

Remediation:

To utilize AWS ECR for Image scanning please follow the steps below:
To create a repository configured for scan on push (AWS CLI)

```
aws ecr create-repository --repository-name $REPO_NAME --image-scanning-
configuration scanOnPush=true --region $REGION_CODE
```

To edit the settings of an existing repository (AWS CLI)

```
aws ecr put-image-scanning-configuration --repository-name $REPO_NAME --
image-scanning-configuration scanOnPush=true --region $REGION_CODE
```

Use the following steps to start a manual image scan using the AWS Management Console.

1. Open the Amazon ECR console at https://console.aws.amazon.com/ecr/repositories.
2. From the navigation bar, choose the Region to create your repository in.
3. In the navigation pane, choose Repositories.
4. On the Repositories page, choose the repository that contains the image to scan.
5. On the Images page, select the image to scan and then choose Scan.

Default Value:

Images are not scanned by Default.

References:

1. https://docs.aws.amazon.com/AmazonECR/latest/userguide/image-scanning.html

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	7.5 Perform Automated Vulnerability Scans of Internal Enterprise Assets Perform automated vulnerability scans of internal enterprise assets on a quarterly, or more frequent, basis. Conduct both authenticated and unauthenticated scans, using a SCAP-compliant vulnerability scanning tool.		●	●
v8	7.6 Perform Automated Vulnerability Scans of Externally-Exposed Enterprise Assets Perform automated vulnerability scans of externally-exposed enterprise assets using a SCAP-compliant vulnerability scanning tool. Perform scans on a monthly, or more frequent, basis.		●	●

Controls Version	Control	IG 1	IG 2	IG 3
v7	3.1 Run Automated Vulnerability Scanning Tools Utilize an up-to-date SCAP-compliant vulnerability scanning tool to automatically scan all systems on the network on a weekly or more frequent basis to identify all potential vulnerabilities on the organization's systems.		●	●
v7	3.2 Perform Authenticated Vulnerability Scanning Perform authenticated vulnerability scanning with agents running locally on each system or with remote scanners that are configured with elevated rights on the system being tested.		●	●

5.1.2 Minimize user access to Amazon ECR (Manual)

Profile Applicability:

- Level 1

Description:

Restrict user access to Amazon ECR, limiting interaction with build images to only authorized personnel and service accounts.

Rationale:

Weak access control to Amazon ECR may allow malicious users to replace built images with vulnerable containers.

Impact:

Care should be taken not to remove access to Amazon ECR for accounts that require this for their operation.

Audit:

Remediation:

Before you use IAM to manage access to Amazon ECR, you should understand what IAM features are available to use with Amazon ECR. To get a high-level view of how Amazon ECR and other AWS services work with IAM, see AWS Services That Work with IAM in the IAM User Guide.
Topics

- Amazon ECR Identity-Based Policies
- Amazon ECR Resource-Based Policies
- Authorization Based on Amazon ECR Tags
- Amazon ECR IAM Roles

Amazon ECR Identity-Based Policies

With IAM identity-based policies, you can specify allowed or denied actions and resources as well as the conditions under which actions are allowed or denied. Amazon ECR supports specific actions, resources, and condition keys. To learn about all of the elements that you use in a JSON policy, see IAM JSON Policy Elements Reference in the IAM User Guide.

Actions

The Action element of an IAM identity-based policy describes the specific action or actions that will be allowed or denied by the policy. Policy actions usually have the same name as the associated AWS API operation. The action is used in a policy to grant permissions to perform the associated operation.

Policy actions in Amazon ECR use the following prefix before the action: ecr:. For example, to grant someone permission to create an Amazon ECR repository with the Amazon ECR CreateRepository API operation, you include the ecr:CreateRepository action in their policy. Policy statements must include either an Action or NotAction element. Amazon ECR defines its own set of actions that describe tasks that you can perform with this service.

To specify multiple actions in a single statement, separate them with commas as follows:

```
"Action": [ "ecr:action1", "ecr:action2"
```

You can specify multiple actions using wildcards (*). For example, to specify all actions that begin with the word Describe, include the following action:

```
"Action": "ecr:Describe*"
```

To see a list of Amazon ECR actions, see Actions, Resources, and Condition Keys for Amazon Elastic Container Registry in the IAM User Guide.

Resources

The Resource element specifies the object or objects to which the action applies. Statements must include either a Resource or a NotResource element. You specify a resource using an ARN or using the wildcard (*) to indicate that the statement applies to all resources.

An Amazon ECR repository resource has the following ARN:

```
arn:${Partition}:ecr:${Region}:${Account}:repository/${Repository-name}
```

For more information about the format of ARNs, see Amazon Resource Names (ARNs) and AWS Service Namespaces.

For example, to specify the my-repo repository in the us-east-1 Region in your statement, use the following ARN:

```
"Resource": "arn:aws:ecr:us-east-1:123456789012:repository/my-repo"
```

To specify all repositories that belong to a specific account, use the wildcard (*):

```
"Resource": "arn:aws:ecr:us-east-1:123456789012:repository/*"
```

To specify multiple resources in a single statement, separate the ARNs with commas.

```
"Resource": [ "resource1", "resource2"
```

To see a list of Amazon ECR resource types and their ARNs, see Resources Defined by Amazon Elastic Container Registry in the IAM User Guide. To learn with which actions you can specify the ARN of each resource, see Actions Defined by Amazon Elastic Container Registry.

Condition Keys
The Condition element (or Condition block) lets you specify conditions in which a statement is in effect. The Condition element is optional. You can build conditional expressions that use condition operators, such as equals or less than, to match the condition in the policy with values in the request.
If you specify multiple Condition elements in a statement, or multiple keys in a single Condition element, AWS evaluates them using a logical AND operation. If you specify multiple values for a single condition key, AWS evaluates the condition using a logical OR operation. All of the conditions must be met before the statement's permissions are granted.
You can also use placeholder variables when you specify conditions. For example, you can grant an IAM user permission to access a resource only if it is tagged with their IAM user name. For more information, see IAM Policy Elements: Variables and Tags in the IAM User Guide.
Amazon ECR defines its own set of condition keys and also supports using some global condition keys. To see all AWS global condition keys, see AWS Global Condition Context Keys in the IAM User Guide.
Most Amazon ECR actions support the aws:ResourceTag and ecr:ResourceTag condition keys. For more information, see Using Tag-Based Access Control.
To see a list of Amazon ECR condition keys, see Condition Keys Defined by Amazon Elastic Container Registry in the IAM User Guide. To learn with which actions and resources you can use a condition key, see Actions Defined by Amazon Elastic Container Registry.

References:

1. https://docs.aws.amazon.com/AmazonECR/latest/userguide/image-scanning.html#scanning-repository

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	5.4 Restrict Administrator Privileges to Dedicated Administrator Accounts Restrict administrator privileges to dedicated administrator accounts on enterprise assets. Conduct general computing activities, such as internet browsing, email, and productivity suite use, from the user's primary, non-privileged account.	●	●	●
v7	4.3 Ensure the Use of Dedicated Administrative Accounts Ensure that all users with administrative account access use a dedicated or secondary account for elevated activities. This account should only be used for administrative activities and not internet browsing, email, or similar activities.	●	●	●

5.1.3 Minimize cluster access to read-only for Amazon ECR (Manual)

Profile Applicability:

- Level 1

Description:

Configure the Cluster Service Account with Storage Object Viewer Role to only allow read-only access to Amazon ECR.

Rationale:

The Cluster Service Account does not require administrative access to Amazon ECR, only requiring pull access to containers to deploy onto Amazon EKS. Restricting permissions follows the principles of least privilege and prevents credentials from being abused beyond the required role.

Impact:

A separate dedicated service account may be required for use by build servers and other robot users pushing or managing container images.

Audit:

Review AWS ECS worker node IAM role (NodeInstanceRole) IAM Policy Permissions to verify that they are set and the minimum required level.
If utilizing a 3rd party tool to scan images utilize the minimum required permission level required to interact with the cluster - generally this should be read-only.

Remediation:

You can use your Amazon ECR images with Amazon EKS, but you need to satisfy the following prerequisites.
The Amazon EKS worker node IAM role (NodeInstanceRole) that you use with your worker nodes must possess the following IAM policy permissions for Amazon ECR.

```
{
    "Version": "2012-10-17",
    "Statement": [
        {
            "Effect": "Allow",
            "Action": [
                "ecr:BatchCheckLayerAvailability",
                "ecr:BatchGetImage",
                "ecr:GetDownloadUrlForLayer",
                "ecr:GetAuthorizationToken"
            ],
            "Resource": "*"
        }
    ]
}
```

Default Value:

If you used eksctl or the AWS CloudFormation templates in Getting Started with Amazon EKS to create your cluster and worker node groups, these IAM permissions are applied to your worker node IAM role by default.

References:

1. https://docs.aws.amazon.com/AmazonECR/latest/userguide/ECR_on_EKS.html

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	5.4 Restrict Administrator Privileges to Dedicated Administrator Accounts Restrict administrator privileges to dedicated administrator accounts on enterprise assets. Conduct general computing activities, such as internet browsing, email, and productivity suite use, from the user's primary, non-privileged account.	●	●	●
v7	4.3 Ensure the Use of Dedicated Administrative Accounts Ensure that all users with administrative account access use a dedicated or secondary account for elevated activities. This account should only be used for administrative activities and not internet browsing, email, or similar activities.	●	●	●

5.1.4 Minimize Container Registries to only those approved (Manual)

Profile Applicability:

- Level 1

Description:

Use approved container registries.

Rationale:

Allowing unrestricted access to external container registries provides the opportunity for malicious or unapproved containers to be deployed into the cluster. Allowlisting only approved container registries reduces this risk.

Impact:

All container images to be deployed to the cluster must be hosted within an approved container image registry.

Audit:

Remediation:

To minimize AWS ECR container registries to only those approved, you can follow these steps:

1. Define your approval criteria: Determine the criteria that containers must meet to be considered approved. This can include factors such as security, compliance, compatibility, and other requirements.
2. Identify all existing ECR registries: Identify all ECR registries that are currently being used in your organization.
3. Evaluate ECR registries against approval criteria: Evaluate each ECR registry against your approval criteria to determine whether it should be approved or not. This can be done by reviewing the registry settings and configuration, as well as conducting security assessments and vulnerability scans.
4. Establish policies and procedures: Establish policies and procedures that outline how ECR registries will be approved, maintained, and monitored. This should include guidelines for developers to follow when selecting a registry for their container images.
5. Implement access controls: Implement access controls to ensure that only approved ECR registries are used to store and distribute container images. This can be done by setting up IAM policies and roles that restrict access to unapproved registries or create a whitelist of approved registries.
6. Monitor and review: Continuously monitor and review the use of ECR registries to ensure that they continue to meet your approval criteria. This can include

regularly reviewing access logs, scanning for vulnerabilities, and conducting periodic audits.

By following these steps, you can minimize AWS ECR container registries to only those approved, which can help to improve security, reduce complexity, and streamline container management in your organization. Additionally, AWS provides several tools and services that can help you manage your ECR registries, such as AWS Config, AWS CloudFormation, and AWS Identity and Access Management (IAM).

Default Value:

Container registries are not restricted by default and Kubernetes assumes your default CR is Docker Hub.

References:

1. https://aws.amazon.com/blogs/opensource/using-open-policy-agent-on-amazon-eks/

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	2.5 Allowlist Authorized Software Use technical controls, such as application allowlisting, to ensure that only authorized software can execute or be accessed. Reassess bi-annually, or more frequently.		●	●
v7	5.3 Securely Store Master Images Store the master images and templates on securely configured servers, validated with integrity monitoring tools, to ensure that only authorized changes to the images are possible.		●	●
v7	13.4 Only Allow Access to Authorized Cloud Storage or Email Providers Only allow access to authorized cloud storage or email providers.		●	●

5.2 Identity and Access Management (IAM)

This section contains recommendations relating to using AWS IAM with Amazon EKS.

5.2.1 Prefer using dedicated EKS Service Accounts (Automated)

Profile Applicability:

- Level 1

Description:

Kubernetes workloads should not use cluster node service accounts to authenticate to Amazon EKS APIs. Each Kubernetes workload that needs to authenticate to other AWS services using AWS IAM should be provisioned with a dedicated Service account.

Rationale:

Manual approaches for authenticating Kubernetes workloads running on Amazon EKS against AWS APIs are: storing service account keys as a Kubernetes secret (which introduces manual key rotation and potential for key compromise); or use of the underlying nodes' IAM Service account, which violates the principle of least privilege on a multi-tenanted node, when one pod needs to have access to a service, but every other pod on the node that uses the Service account does not.

Audit:

For each namespace in the cluster, review the rights assigned to the default service account and ensure that it has no roles or cluster roles bound to it apart from the defaults.
Additionally ensure that the automountServiceAccountToken: false setting is in place for each default service account.

Remediation:

With IAM roles for service accounts on Amazon EKS clusters, you can associate an IAM role with a Kubernetes service account. This service account can then provide AWS permissions to the containers in any pod that uses that service account. With this feature, you no longer need to provide extended permissions to the worker node IAM role so that pods on that node can call AWS APIs.
Applications must sign their AWS API requests with AWS credentials. This feature provides a strategy for managing credentials for your applications, similar to the way that Amazon EC2 instance profiles provide credentials to Amazon EC2 instances. Instead of creating and distributing your AWS credentials to the containers or using the Amazon EC2 instance's role, you can associate an IAM role with a Kubernetes service account. The applications in the pod's containers can then use an AWS SDK or the AWS CLI to make API requests to authorized AWS services.
The IAM roles for service accounts feature provides the following benefits:

- Least privilege — By using the IAM roles for service accounts feature, you no longer need to provide extended permissions to the worker node IAM role so that pods on that node can call AWS APIs. You can scope IAM permissions to a service account, and only pods that use that service account have access to those permissions. This feature also eliminates the need for third-party solutions such as kiam or kube2iam.

- Credential isolation — A container can only retrieve credentials for the IAM role that is associated with the service account to which it belongs. A container never has access to credentials that are intended for another container that belongs to another pod.
- Audit-ability — Access and event logging is available through CloudTrail to help ensure retrospective auditing.

To get started, see list text hereEnabling IAM roles for service accounts on your cluster. For an end-to-end walkthrough using eksctl, see Walkthrough: Updating a DaemonSet to use IAM for service accounts.

References:

1. https://docs.aws.amazon.com/eks/latest/userguide/iam-roles-for-service-accounts.html
2. https://docs.aws.amazon.com/eks/latest/userguide/iam-roles-for-service-accounts-cni-walkthrough.html
3. https://aws.github.io/aws-eks-best-practices/security/docs/iam/#scope-the-iam-role-trust-policy-for-irsa-to-the-service-account-name

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	12.8 Establish and Maintain Dedicated Computing Resources for All Administrative Work Establish and maintain dedicated computing resources, either physically or logically separated, for all administrative tasks or tasks requiring administrative access. The computing resources should be segmented from the enterprise's primary network and not be allowed internet access.			●
v7	4.3 Ensure the Use of Dedicated Administrative Accounts Ensure that all users with administrative account access use a dedicated or secondary account for elevated activities. This account should only be used for administrative activities and not internet browsing, email, or similar activities.	●	●	●

5.3 AWS EKS Key Management Service

This section contains recommendations relating to using AWS EKS Key Management.

5.3.1 Ensure Kubernetes Secrets are encrypted using Customer Master Keys (CMKs) managed in AWS KMS (Manual)

Profile Applicability:

- Level 1

Description:

Encrypt Kubernetes secrets, stored in etcd, using secrets encryption feature during Amazon EKS cluster creation.

Rationale:

Kubernetes can store secrets that pods can access via a mounted volume. Today, Kubernetes secrets are stored with Base64 encoding, but encrypting is the recommended approach. Amazon EKS clusters version 1.13 and higher support the capability of encrypting your Kubernetes secrets using AWS Key Management Service (KMS) Customer Managed Keys (CMK). The only requirement is to enable the encryption provider support during EKS cluster creation.

Use AWS Key Management Service (KMS) keys to provide envelope encryption of Kubernetes secrets stored in Amazon EKS. Implementing envelope encryption is considered a security best practice for applications that store sensitive data and is part of a defense in depth security strategy.

Application-layer Secrets Encryption provides an additional layer of security for sensitive data, such as user defined Secrets and Secrets required for the operation of the cluster, such as service account keys, which are all stored in etcd.

Using this functionality, you can use a key, that you manage in AWS KMS, to encrypt data at the application layer. This protects against attackers in the event that they manage to gain access to etcd.

Audit:

For Amazon EKS clusters with Secrets Encryption enabled, run the AWS CLI command:

```
aws eks describe-cluster --name=<cluster-name>
```

From the output of the command, search if the following configuration exist with valid AWS `keyArn` :